

Data Exploration Guide for Linear Regression

Boston Housing Dataset

Step 1: Check Data Shape and Missing Values

Code:

```
print(df.shape)
print(df.info())
print(df.isnull().sum())
```

What to look for and why:

df.shape: Should show (506, 14) for Boston data — 506 observations, 14 columns

df.info(): Confirms all columns are numeric (float64 or int64). If you see 'object' type, you have text data that needs encoding

df.isnull().sum(): Should show 0 missing values in all columns. If there are nulls, linear regression won't work without handling them (imputation or removal)

Why it matters: Linear regression assumes complete data. Missing values bias your model. Also, mixed data types can cause computation errors.

Step 2: Descriptive Statistics

Code:

```
df.describe()
```

What to look for and why:

Mean vs. median: Are they close? If very different, data is skewed (skewed distributions violate normality assumption)

Min/max values: Are they reasonable? Large outliers can pull the regression line and make predictions unreliable

Standard deviation: Features with very different scales may need normalization (optional for linear regression, but helpful for interpretation)

25th/75th percentiles: Look for data concentrated in a small range vs. spread out range — affects model sensitivity

Why it matters: Linear regression assumes roughly normal distributions. Skewed data or extreme outliers can violate this and lead to poor predictions.

Step 3: Correlations

Code:

```
corr_matrix = df.corr()
print(corr_matrix)
```

What to look for and why:

Correlation with MEDV (target): Which features have the strongest correlation (closest to 1.0 or -1.0)? These are your most important predictors. For example: RM (rooms) likely has high positive correlation with house price; LSTAT (lower status %) likely has high negative

correlation

Correlations between features: Look for correlations above 0.8 or below -0.8 between different features. This is **multicollinearity** — when two features measure nearly the same thing. This makes the model unstable and hard to interpret

Any surprising patterns: Features you thought were important but have low correlation with the target

Why it matters: High correlation with target = strong predictor (useful features). High correlation between features = multicollinearity (causes inflated coefficients and unstable predictions). Linear regression assumes features are independent from each other

Step 4: Visualize Correlations with a Heatmap

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

What to look for and why:

Dark red cells: Strong positive correlations

Dark blue cells: Strong negative correlations

Light/white cells: Weak or no correlation

Bottom-right corner (MEDV row/column): Which features show the darkest red/blue?
Those are your best predictors

Blocks of dark colors between features: Indicates multicollinearity — watch out for these

Why it matters: Visual patterns are easier to spot than numbers. You can quickly identify which features drive the target and where multicollinearity problems exist.

Step 5: Check Feature-Target Relationships

Code:

```
# Scatter plot for a key feature vs. target
plt.scatter(df['RM'], df['MEDV'])
plt.xlabel('RM (Number of Rooms)')
plt.ylabel('MEDV (House Price)')
plt.title('Rooms vs. House Price')
plt.show()

# Or do this for multiple features
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].scatter(df['RM'], df['MEDV'])
axes[0, 0].set_title('RM vs MEDV')
axes[0, 1].scatter(df['LSTAT'], df['MEDV'])
axes[0, 1].set_title('LSTAT vs MEDV')
plt.tight_layout()
plt.show()
```

What to look for and why:

Linear pattern: Points should roughly follow a straight line. If they form a curve, the relationship is non-linear (linear regression might not fit well)

Spread around the line: If the scatter widens (more spread on the right than left), this violates the homoscedasticity assumption

Outliers: Points far from the main pattern. These can skew your regression line

Why it matters: Linear regression assumes linear relationships. Non-linear patterns mean you may need feature transformation or a different model entirely.

Summary: What You're Actually Checking

You're validating the **assumptions** of linear regression:

- ✓ Data completeness (no nulls)
- ✓ Reasonable distributions (not heavily skewed)
- ✓ Important features correlate with target (predictive power)
- ✓ Features don't correlate too much with each other (multicollinearity)
- ✓ Relationships are linear (not curved)

If most of these check out, your data is ready for linear regression.